

Day- 1

Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR





Today's Agenda

- Programming Basics & Syntax
- Introduction to Java
- Input and Output Operations
- Variables
- Data Types



Programming Basics & Syntax

Q. What is a Program?
Ans. A program is a set of instructions written in a programming language that tells a computer what to do and how to do it.

Q. What is Programming?
Ans. Programming is the process of giving instructions to a computer to perform specific tasks or solve problems using a programming language that the computer can understand.

Examples: C, C++, Java, Python, etc.

Q. Why is Programming Important?
Ans.

- To create software and applications.
- To automate tasks.
- To solve problems logically.
- To make machines work efficiently.

Programming Basics & Syntax

Q. What is Software?

Ans. Software is a collection of programs that controls the working of computer hardware and helps users perform tasks.

- Software is a program that tells a computer or mobile device what work to do.

Example: Music player application
Work: Plays, pauses, and changes songs.

❖ **Types of Software**

1. System Software:- System software helps run and control the computer.
Examples: Windows, Linux, Android

2. Application Software:- Application software is used to perform specific tasks.
Examples: MS Word, Browser, Calculator, etc.

Programming Basics & Syntax

Q. What is a Compiler and Interpreter?

Ans. Both compiler and interpreter are used to convert program code into machine language so that the computer can understand and execute it.

❖ **Compiler**

A compiler converts the entire program at once and shows errors after checking the complete program.

Used in: C, C++, Java

❖ **Interpreter**

An interpreter converts the program line by line and stops immediately if an error is found.

Used in: Python, JavaScript



Programming Basics & Syntax

Q. What is an Algorithm?

Ans. An algorithm is a step-by-step procedure to solve a problem or perform a task.

Example: Using an ATM

i. Insert the ATM card
ii. Enter the PIN
iii. Select the transaction
iv. Enter the amount
v. Collect the cash
vi. Take the receipt
vii. Remove the card





Introduction to Java





Introduction to Java

Q. What is Java?

Ans. Java is a simple, high-level, secure, platform-independent, multithreaded, and object-oriented programming language. It was developed by James Gosling at Sun Microsystems in the early 1990s and officially released in 1995. Java is mainly used to develop network-based and high-performance applications.

Q. Why is Java popular?

Ans. Java is popular because:

- It is easy to learn and use.
- It is secure and reliable. +
- It is used in many devices and platforms.
- t is widely used by big companies and organizations.

❖ **Examples of Java Applications:**

- Mobile Applications
- Web Applications
- Desktop Applications
- Banking Software
- Enterprise Applications





Introduction to Java

❖ Features of Java

- Platform Independent
- Secure
- Robust
- Object-Oriented
- Multithreaded

JVM, JRE and JDK

❖ JVM (Java Virtual Machine)

- JVM is a virtual machine that runs Java bytecode.
- It is the engine that executes Java programs.
- JVM converts bytecode into machine code.
- It makes Java platform independent.
- JVM helps in running Java programs on different operating systems.





Introduction to Java

❖ **JRE (Java Runtime Environment)**

- JRE provides the environment required to run Java programs.

  JRE = JVM + Libraries
- It is used to run Java programs.
- It is not used to write or develop code.

Example:

Just like a TV needs electricity to run, Java programs need JRE to run.

❖ **JDK (Java Development Kit)**

- JDK provides tools to develop Java applications.

  JDK = JRE + Development Tools
- It is used by programmers to write, compile, and run Java programs.

Example:

Just like a mechanic needs tools to repair a machine, a programmer needs JDK to write code.





Introduction to Java

❖ **Skeleton Of Java Program**

```
import java.lang.*;

 Class First {
            public static void main(String[] args) {
                    System.out.println("Hello, World...");
             }
}
```

- Java.lang.*; -- It is a package
- main() – method to execute the program
- static-- if you want to use class without creating an object we need to use static.
- System.out.println();-- It is a method to print any thing.
- string[] args-- Command line arguments used to take input from command.

Note: - File name and class name should be same.





Input and Output Operations





Input and Output Operations

❖ **Reading from keyboard**

- Reading from keyboard means taking input from the user while the program is running.
- Java reads keyboard input using the “Scanner” class with “system.in”

Example:

```
import java.lang.*;
import java.util.Scanner.*;
Class Read{
        public static void main(String[] args){
        Scanner sc= new Scanner(System.in);
        System.out.println(“Enter your name: “);
        String name= sc.nextLine();
        System.out.println(“Name: “+ name);
        }
}
```



Input and Output Operations

❖ Java Input Methods (Scanner Class)

- **nextInt()** – Used to read whole numbers (integers).
- **nextFloat()** – Used to read decimal numbers.
- **nextDouble()** – Used to read more accurate decimal values.
- **next()** – Reads only one word and stops when it finds a space.
- **nextLine()** – Reads the entire line, including spaces.

❖ Output Operations in Java

Output operations are used to display data on the screen. In Java, output is mainly performed using **System.out**.

❖ Methods of Output:

- **System.out.print()** – Prints the output and keeps the cursor on the same line.
- **System.out.println()** – Prints the output and moves the cursor to the next line.



Variables

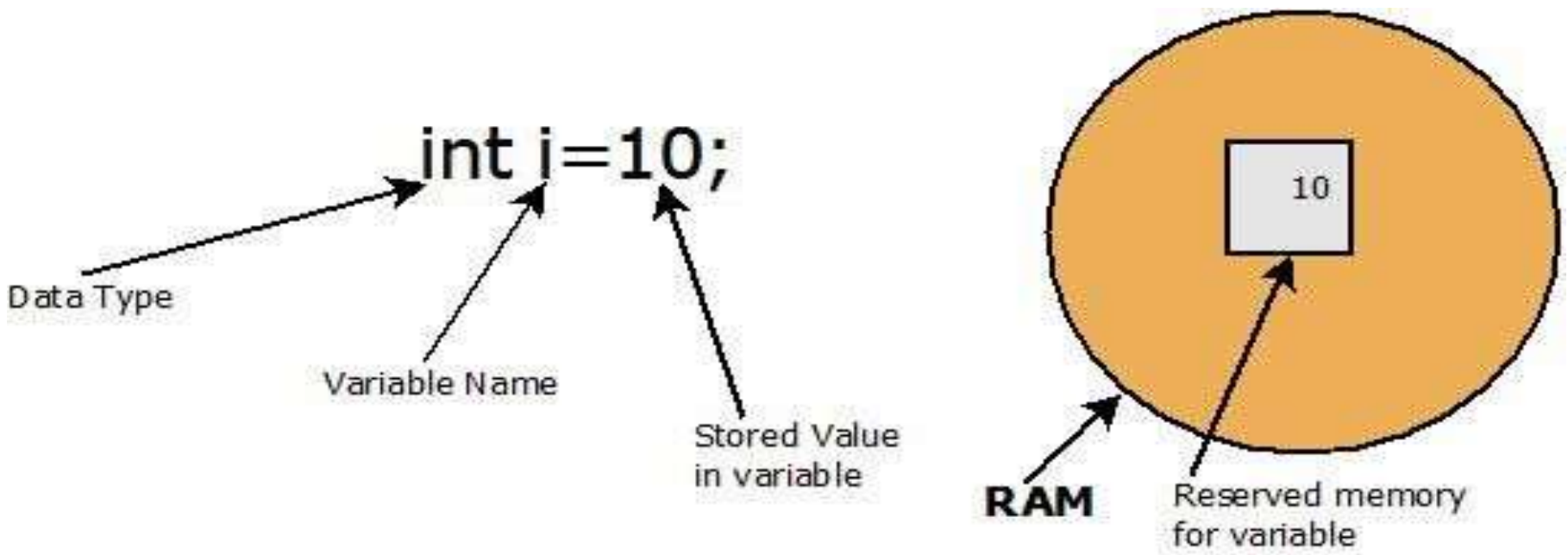


Variables

- A variable is the name of a memory location where data is stored.

Syntax:

datatype variableName = Value;





Variables

❖ **Rules for Naming Variables in java.**

- Case sensitive [age Vs Age]
- Starting character [a, A, _ , $]
- Subsequent character [0 to 9, _, $]
- No Reserved keywords [Class, static etc]
- Length [No limit but meaningful names needs]
- Naming Convensitions: CamelCase



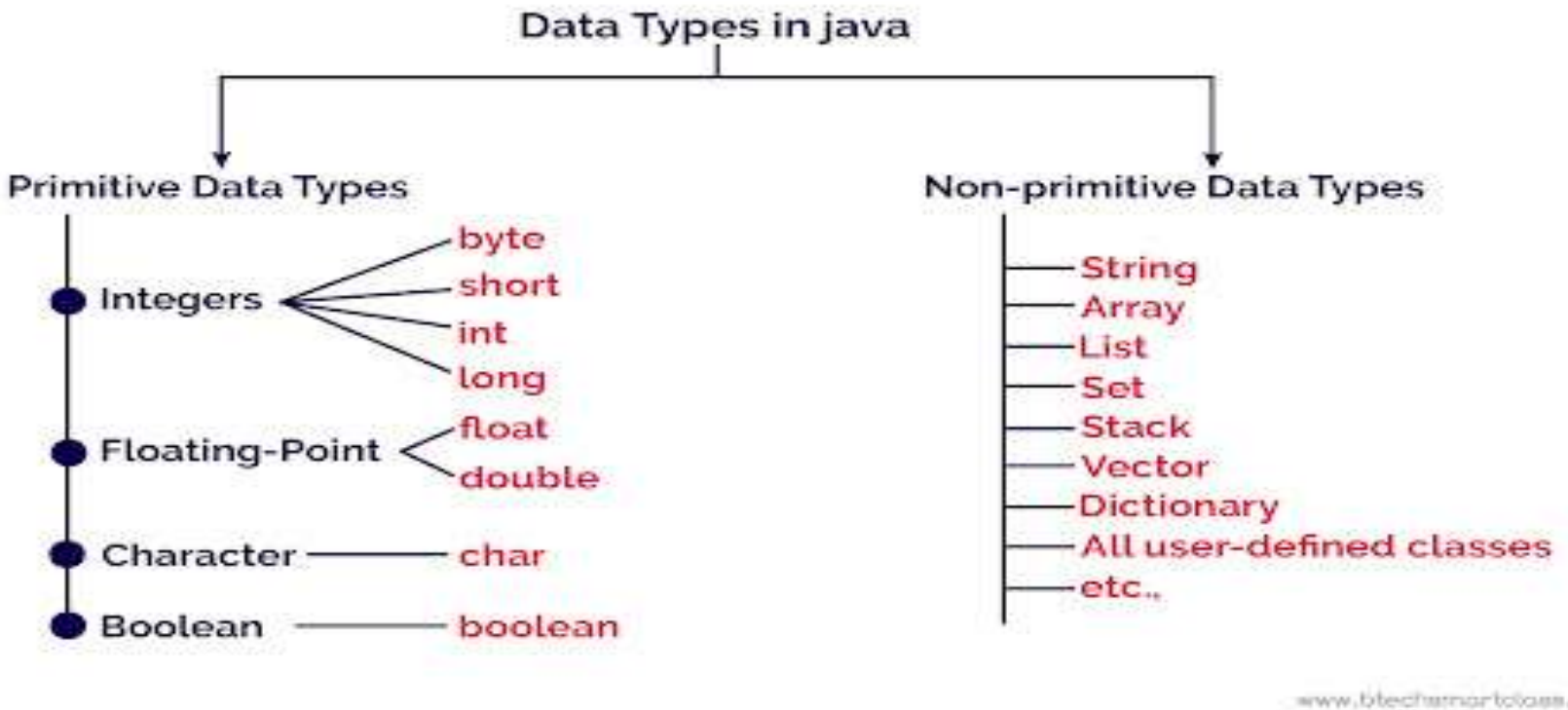


Data Types



Date Types

- Data types tell us what kind of data can be stored in a variable, such as numbers, text, or true/false values.





Exercise

1. Write a program to print Hello Students on the screen.
2. Write a Program to print Hello on the first line and Students in the second line.
3. Write a program to print “AmanXcel” on the screen.
4. Write a program to calculate the sum of two numbers.
5. Write a program to calculate the area of a circle.
6. Write a program to calculate the volume of a Cuboid.
7. Write a program to calculate the simple interest
8. Write a program to calculate the area of a rectangle
9. Write a program to calculate the average of three numbers
10. Write a program to calculate the square of a given number.
11. Write a program to find last digit of a given number
12. Write a program to swap values of two int variables
13. Write a program to swap values of two int variables without using third variable
14. Assume price of 1 USD is INR 76.23. Write a program to take the amount in INR and convert it into USD.
15. Write a program to make the last digit of a number stored in a variable as zero. (Example - if x=2345 then make it x=2340)





Day- 2

Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR





Today's Agenda

- Operators
- Conditional Statements
- Exercises





Operators

Q. What is an Operator?
Ans. An operator is a special symbol that performs some operation on given values or variables.

❖ **Types of Operators:**

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Assignment Operators
5. Unary Operators
6. Bitwise Operators





Operators

1. Arithmetic Operators:

Arithmetic operators are used to perform mathematical operations on variables and values.

Examples: +, -, *, /, %

2. Relational Operators:

Relational operators are used to compare two values or variables.

Examples: ==, !=, >, <, >=, <=

3. Logical Operators:

Logical operators are used to combine multiple conditions and return true or false.

Examples: AND (&&), OR (||), NOT (!)

4. Assignment Operators:

Assignment operators are used to assign values to variables.

Examples: =, +=, -=, *=, /=, %=





Operators

5. Unary (Increment/Decrement) Operators:
Unary operators are operators that work on a single operand.

Examples: ++, --

6. Bitwise Operators: Bitwise operators are used to perform operations on individual bits (binary values).

Examples: AND (&), OR (|), XOR (^), NOT (~), Left Shift (<<), Right Shift (>>)





Conditional Statement

❖ **Conditional Statements:**

A conditional statement is a control structure that executes a block of code only if a specified condition is true.

❖ **Types of Conditional Statements:**

i. If Statement
ii. If-Else Statement
iii. If-Else-If Ladder
iv. Nested If-Else
v. Ternary Operator
vi. Switch Statement

1. If Statement:

An if statement is used to check a condition. If the condition is true, the block of code executes.

Syntax:

```
if (condition) {
        // code to execute
}
```



Conditional Statement

2. If-Else Statement:

An if-else statement is used to check a condition and execute one block if the condition is true; otherwise, it executes another block.

Syntax:

```
if (condition) {
        // code to execute if the condition is true
} else {
        // code to execute if the condition is false
}
```



Conditional Statement

3. Else-If Ladder:

An else-if ladder is used to check multiple conditions one by one. If one condition is true, its corresponding block of code executes.

Syntax:

```
if (condition1) {
        // code to execute if condition1 is true
} else if (condition2) {
        // code to execute if condition2 is true
} else if (condition3) {
        // code to execute if condition3 is true
} else {
        // code to execute if none of the conditions are true
}
```



Conditional Statement

4. Nested If-Else:

Nested if-else means using an if-else statement inside another if or else block.

Syntax:

```
if (condition1) {
        if (condition2) {
        // code to execute if both conditions are true
        } else {
        // code to execute if condition2 is false
          }
} else {
   // code to execute if condition1 is false
}
```



Conditional Statement

5. Ternary Operator:

A ternary operator is a one-line conditional expression used to assign a value based on a condition.

Syntax:

```
variable = (condition) ? value_if_true : value_if_false;
```

Example:

```
public class Main {
    public static void main(String[] args) {
        int days = 35;
        String status = (days >= 30) ? "Regular" : "Irregular";

        System.out.println(status);
    }
}
```





Conditional Statement

6. Switch Statement:

A switch statement is used to select one block of code from multiple options based on the value of an expression.

Syntax:

```
switch (expression) {
case value1:
        // code to execute
         break;

 case value2:
         // code to execute
         break;

default:
        // code to execute if no case matches
}
```





Exercise

1. Write a program to add two numbers.
2. Write a program to find the remainder when one number is divided by another.
3. Write a program to calculate the area of a rectangle.
4. Write a program to swap two numbers without using a third variable.
5. Write a program to calculate simple interest.
6. Write a program to check whether two numbers are equal or not.
7. Write a program to find the greater number between two numbers.
8. Write a program to check whether a number is positive, negative, or zero.
9. Write a program to check if a person is eligible to vote (age ≥ 18).
10. Write a program to check if a student has passed (marks ≥ 40 and attendance ≥ 75).
11. Write a program to demonstrate increment (++) and decrement (--) operators.
12. Write a program to check whether a number is positive.
13. Write a program to check whether a person is eligible for voting.
14. Write a program to check whether a number is divisible by 5.
15. Write a program to check whether a number is even or odd.
16. Write a program to find the greater of two numbers.
17. Write a program to check whether a student passed or failed.





Exercise

18. Write a program to find the largest among three numbers.
19. Write a program to display grades based on marks (A, B, C, Fail).
20. Write a program to check if a year is a leap year.
21. Write a program to check whether a number is even or odd using the ternary operator.
22. Write a program to find the greater between two numbers using the ternary operator.
23. Write a program to display the day name based on a number (1–7).
24. Write a calculator program using switch (+, -, *, /). Write a program to display month names based on month number (1–12).





Day- 3

Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR





Loops in Java

- A loop is a control structure that repeats a set of statement as long as a given condition is true.

- Loops in Java are used to repeat a block of code multiple times until a condition is met. They're fundamental for tasks like iterating over arrays, processing data, or repeating actions.

❖ **Types of loops in Java**

1. for loop
2. while loop
3. do-while loop

1. for loop: Used when you know how many times you want to run the loop.

Syntax:

```
for( initialization, condition, update){
        // code
        //code
}
```





Loops in Java

2. While Loop:

A while loop is used when you don’t know the exact number of iterations, and the loop depends on a condition.

Syntax:

```
while(condition) {
        // code to execute
}
```

Example:

```
int i = 1;

while(i <= 5) {
    System.out.println(i);
    i++;
}
```





Loops in Java

3. Do-While Loop:

A do-while loop executes the code at least once, even if the condition is false, because the condition is checked after execution.

Syntax:

```
do {
        // code to execute
} while(condition);
```

Example:

```
int i = 1;

do {
        System.out.println(i);
        i++;
} while(i <= 5);
```





Loops in Java

1. break keyword

The break keyword is used to stop the loop immediately.

Example:

```
for (int i = 1; i <= 5; i++) {
        if (i == 3) {
        break;
        }
System.out.println(i); // 1 2
}
```

2. continue keyword

The continue keyword is used to skip the current iteration and move to the next iteration.

Example:

```
for (int i = 1; i <= 5; i++) {
      if (i == 3) {
      continue;
     }
  System.out.println(i); // 1 2 4 5
}
```





Exercise

1. Write a program to print “AmanXcel” 10 times on the screen.
2. Write a program to print the first 10 natural numbers.
3. Write a program to print the first 10 natural numbers in reverse order.
4. Write a program to print the first 10 even natural numbers.
5. Write a program to print the first 10 even natural numbers in reverse order.
6. Write a program to print the first 10 odd natural numbers
7. Write a program to print the squares of first 10 natural numbers
8. Write a program to print the cubes of first 10 natural numbers
9. Write a program to print the squares of first 10 natural numbers in reverse order
10. Write a program to print the cubes of the first 10 natural numbers in reverse order.
11. Write a program to print “AmanXcel” N times on the screen.
12. Write a program to print the first N natural numbers.
13. Write a program to print the first N natural numbers in reverse order.
14. Write a program to print the first N even natural numbers.
15. Write a program to print the first N even natural numbers in reverse order.





Exercise

16. Write a program to print the first N odd natural numbers
17. Write a program to print the squares of first N natural numbers
18. Write a program to print the cubes of first N natural numbers
19. Write a program to print the squares of first N natural numbers in reverse order
20. Write a program to print the cubes of the first N natural numbers in reverse order.
21. Write a program to calculate sum of first N natural numbers
22. Write a program to calculate sum of first N even natural numbers
23. Write a program to calculate sum of first N odd natural numbers
24. Write a program to calculate sum of squares of first N natural numbers
25. Write a program to calculate sum of cubes of first N natural numbers
26. Write a program to calculate factorial of a number
27. Write a program to count digits in a given number
28. Write a program to check whether a given number is a Prime number or not





Day- 4

Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR





Nested Loop

Q. What is nested Loop?
Ans. A **nested loop** means placing one loop inside another loop.

- The first loop is called the **Outer Loop**
- The loop inside it is called the **Inner Loop**

The inner loop executes completely for every single iteration of the outer loop.

Example: Think of a school:
There are **3 classrooms**
Each classroom has **5 students**





Nested Loop

❖ **Syntax in Java**

```
for(initialization; condition; increment/decrement) {    // Outer loop

        for(initialization; condition; increment/decrement) {    // Inner loop

                // statements
         }
}
```

❖ **Example:**

```
for(int i=1; i<=3; i++) {
         for(int j=1; j<=4; j++) {
                System.out.print("* ");
        }
        System.out.println();
}
```





Nested Loop

❖ **Important Points**

- Outer loop runs first.
- Inner loop runs completely every time.
- Used for patterns.
- Used in matrices.
- Used in sorting algorithms.





Exercise

1. Print stars in rectangle

```
* * * *
* * * *
* * * *
```

2. Print numbers

```
1 2 3
1 2 3
1 2 3
```

3. Print row number

```
1 1 1
2 2 2
3 3 3
4 4 4
```

4. Triangle pattern

```
*
* *
* * *
* * * *
```





Nested Loop

5. Reverse triangle

```
* * * *
* * *
* *
*
```

6. Number Triangle

```
1
1 2
1 2 3
1 2 3 4
```

7. Repeat Row Number

```
1
2 2
3 3 3
4 4 4 4
```





Day- 5

Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR





Functions in Java

Q. What is Function?

Ans. In Java, a function is usually called a method. It is a block of code designed to perform a specific task and can be reused whenever needed.

Q. Why do we use methods in Java?

Ans:

- To avoid repetition of code.
- To make the code easier to understand.
- To divide a big problem into smaller parts.

Syntax:

```
returnType methodName(Parameter){
        // method body
        //
}
```





Functions in Java

Example:

```
void printTable(){
        for(int i=1; i<=10; i++){
        System.out.println(2*i);
        }
}
```

❖ **Call/Invoke Method**

```
void main(){
        System.out.println("Hi");
        solve();// function call
        System.out.println("Bye");
}
void solve(){ // definition / declaration
        System.out.println("Aman");
}
```





Functions in Java

❖ **Parameters vs No Parameters**

```
void printSum(int x, int y){ //x and y are Parameters
        System.out.println(x+y);
}
```

- A parameters is a value that you pass to a method so that it can use that data to perform a task.

```
void printSum( ){
        int a =1;
        int b=2;
        System.out.println(a+b);
}
```

- A method that does not take any input is called a method without parameters.





Functions in Java

❖ **Void vs Non-Void Methods in Java**

- A **void method** is a method that does **not return** any value.
- A **non-void method** is a method that **returns** a value.

Example:

```
void main(){
        int result= add(10, 15);
        System.out.println(result);
}

int add(int p, int q){
        int sum= p+q;
        return sum;
}
```





Functions in Java

❖ Method Overloading

- Method overloading means creating multiple methods with the **same name** but **different parameters** in the same class.
- It helps improve code readability and reusability

Example:

```
class Main{
        void add(int a, int b){
              System.out.println(a+n);
         }

         void add( int a, int b, int c){
              System.out.println(x+y+z);
         }
          public static void main(String [] args){
              Main obj = new Main();
              obj.add(5, 3);
              obj.add(2,4,6);
           }
}
```





Functions in Java

❖ Scoping of Variables in java

1. **Local Variable:**
   A variable declared inside a method is called a **local variable**. It can be accessed and used only within that method.

Example:

```
class Demo{
        void show(){
                int x=10; // local Variable
                System.out.println(x);
        }
        public static void main(String[] args){
                Demo obj= new Demo();
                obj.show();
        }
}
```





Functions in Java

❖ **Scoping of Variables in java**

2. Instance Variable (Global Variable)

A variable declared inside a class but outside methods is called an **instance variable**. It can be accessed by all methods of that class.

Example:

```
class Demo{
        int x=10;// Instance Varibale/Global Variable
        void display(){
                System.out.println(x);
        }
        void show(){
                System.out.println(x);
        }
}
```





Exercise

1. Create a method to add two numbers using parameters.
2. Create a method to find the area of a rectangle using parameters.
3. Create a method to find the simple interest using parameters.
4. Write a method that returns the sum of two numbers.
5. Write a method that returns the largest of two numbers.
6. Write a method that returns the factorial of a number.
7. Create two add() methods (2 parameters and 3 parameters).
8. Create overloaded area() methods for square and rectangle.





Exercise

Technical Test 1:
syntax, conditionals, loops, functions





Day- 6

Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR





Today's Agenda

- Array Basics
- Traversal of an Array
- Updating an Array Element
- Insertion in an Array
- Deletion in an Array





Arrays in Java

Q. What is an Array?

Ans. An array is a data structure that stores multiple elements of the same data type in **contiguous** memory locations. Each element can be accessed using its **index**.

Example:

```
int[] numbers = {10, 20, 30, 40, 50};
```

❖ **Declaration of an Array:** Declaration means defining the **data type** and **name** of an array.

Syntax:

```
int[] arr;
```

Here:

int - data type of the array elements
arr - array name





Arrays in Java

❖ **Allocation of an Array:** Allocation means reserving memory for the array elements.

Syntax:

```
arr = new int[5];
```

➢ Here, memory is created for **5 elements**.

❖ **Initialization of an Array:** Initialization means assigning values to the array elements.

Example:

```
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;
arr[3] = 40;
arr[4] = 50;
```

➢ Array indexing starts from **0 (zero)**.





Arrays in Java

❖ **for each loop:**

```
for(int val: arr){
        System.out.println(val);
}
```

❖ **Taking Input in an Array:**

Step 1: Import the Scanner class

```
import java.util.*;
```

Step 2: Create a Scanner object

.

```
Scanner sc = new Scanner(System.in);
```

Step 3: Declare and allocate the array

```
int[] arr = new int[5];
```





Arrays in Java

Step 4: Take input using a loop

```
for (int i = 0; i < arr.length; i++) {
        arr[i] = sc.nextInt();
}
```

Note: Array indexing starts from 0, so the loop condition should be i < arr.length, not i <= arr.length





Arrays in Java

❖ **Traversal of an Array:** Traversal means visiting and accessing each element of an array one by one.

Example:

```
int[] arr = {10, 20, 30, 40, 50};

for (int i = 0; i < arr.length; i++) {
        System.out.println(arr[i]);
}
```

❖ **Updating an Array Element:** Updating means changing the value of an existing element using its index.

Example:

```
int[] arr = {10, 20, 30, 40, 50};

arr[2] = 100;   // Updating index 2
System.out.println(arr[2]);
```





Arrays in Java

❖ **Deletion in an Array:** Deletion means removing an element from a specific position by shifting elements to the left.

Example:

```
int[] arr = {10, 20, 30, 40, 50};
int pos = 2;

for (int i = pos; i < arr.length - 1; i++) {
        arr[i] = arr[i + 1];
}
```





Arrays Exercise

1. Write a program to store 5 integers in an array and print them.
2. Write a program to print all elements of an array using a loop.
3. Write a program to print array elements in reverse order.
4. Write a program to find the length of an array.
5. Write a program to print only even numbers from an array.
6. Write a program to print only odd numbers from an array.
7. Write a program to calculate the sum of all elements in an array.
8. Write a program to calculate the average of array elements.
9. Write a program to find the largest element in an array.
10. Write a program to find the smallest element in an array.
11. Write a program to check whether a given number exists in an array or not.
12. Write a program to replace all negative numbers with 0.
13. Write a program to increase each array element by 5.
14. Write a program to check whether a given number exists in an array or not.
15. Write a program to search an element in an array.
16. Write a program to count even and odd numbers in an array.
17. Write a program to find the second largest element in an array.
18. Write a program to find duplicate elements in an array.
19. Write a program to sort an array in ascending order.
20. Write a program to sort an array in descending order.





Arrays Exercise

25. Write a program to count negative numbers in an array.
26. Write a program to find the difference between largest and smallest element.
27. Write a program to count positive numbers in an array.
28. Write a program to print elements at odd index positions.
29. Write a program to print alternate elements of an array.
30. Write a program to delete the first element of an array.
31. Write a program to delete the last element of an array.
32. Write a program to print array elements using:
    - for loop
    - while loop
    - for-each loop





Arrays Exercise

Dry Run need to solve
Write a program to find duplicate elements in an array.
Write a program to find the second largest element in an array.





Day- 7&8

Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR





Time and Space Complexity

Q. What is Time Complexity?
Ans. Time Complexity tells us how much time a program takes to run as the input size increases.

- How fast or slow is the program when data becomes large?

Example:

Imagine searching for a student in a classroom.

- If students are standing randomly, you may check one by one --- takes more time.
- If students are standing in roll-number order, you can find quickly.

- So different methods take different amounts of time.





Time and Space Complexity

Q. What is Space Complexity?
Ans. Space Complexity tells us how much memory (space) a program uses while running.

- How much extra storage does the program need?

Example:

Suppose you are packing clothes.

- Using only one bag --- less space.
- Using extra bags and boxes --- more space.

- Programs also use memory like this.





Time and Space Complexity

Q. Why Do We Need Time and Space Complexity?
Ans. We use it to:

1. **Compare programs** : Which solution is better?
2. **Handle large data** : A program may work for 10 numbers but fail for 10 lakh numbers.
3. **Write efficient code** : Faster programs give better performance.
4. **Save memory:** Important in mobile apps, games, and embedded systems.



Time and Space Complexity

❖ **Example of Time Complexity**

Example 1: Printing Numbers

```
for(int i = 1; i <= n; i++) {
        System.out.println(i);
}
```

- If n = 5, loop runs 5 times.
- If n = 100, loop runs 100 times.

So time increases with n.

Time Complexity: $O(n)$
Meaning: Time grows linearly with input size.



Time and Space Complexity

Example 2: Nested Loop

```
for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n; j++) {
                System.out.println(i + " " + j);
         }
}
```

If n = 3

Inner loop runs 3 times for each outer loop.

Total = 3 × 3 = 9

Time Complexity: $O(n^2)$





Time and Space Complexity

❖ **Example of Space Complexity**

Example 1

```
int a = 10;
int b = 20;
```

➢ Only fixed variables are used.

Space Complexity: $O(1)$ (Constant space)

Example 2

```
int[] arr = new int[n];
```

➢ Array size depends on n.

If n increases, memory also increases.

Space Complexity: $O(n)$



Time and Space Complexity

❖ **Common Time Complexities**

Complexity	Meaning	Real-Life Idea
O(1)	Constant Time	Taking first book from table
O(n)	Linear Time	Checking students one by one
$O(n^2)$	Quadratic Time	Every student shaking hands with every student
O(log n)	Very Fast	Searching in sorted phonebook



Day- 9

Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR



2D Array

Q. What is a 2D Array in Java?

Ans. A **2D Array** is an array of arrays. It stores data in the form of **rows and columns** like a table or matrix.

❖ **Example:**

Think about a classroom seating arrangement.

❖ **Syntax of 2D Array**

```
datatype[][] arrayName = new datatype[rows][columns];
```

❖ **Example:**

```
int[][] arr = new int[3][3];
```

This creates:

3 rows

3 columns



2D Array

❖ **Visual Representation**

```
int[][] arr = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
};
```

❖ **Accessing Elements**

Syntax:

```
arrayName[row][column]
```

Example:

```
System.out.println(arr[1][2]);
```





2D Array

Taking Input in 2D Array

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int[][] arr = new int[2][3];
        //Taking Input from the user

        for(int i = 0; i < arr.length; i++) {

            for(int j = 0; j < arr[i].length; j++) {

                arr[i][j] = sc.nextInt();
            }
        }
```





2D Array

```
// printing the element
        for(int i = 0; i < arr.length; i++) {

            for(int j = 0; j < arr[i].length; j++) {

                System.out.print(arr[i][j] + " ");
            }

            System.out.println();
        }
```



2D Array

Q. Why Do We Use 2D Arrays?

Ans. We use them when data is in:

- Matrix form
- Table format
- Grid format

Examples:

- Chess board
- Excel sheet
- Student marks table
- Game boards
- Image pixels



2D Array

❖ **Time Complexity of Traversing 2D Array**

If matrix size is n × m

```
for(int i=0; i<n; i++) {
        for(int j=0; j<m; j++) {

 }
}
```

Time Complexity: $O(n \times m)$
Because every element is visited once.

❖ **Space Complexity**

For storing matrix:

```
int[][] arr = new int[n][m];
```

Space Complexity: $O(n \times m)$





2D Arrays Exercise

1. Write a program to print all element of 2D array.
2. Write a program to find sum of all elements
3. Write a program to find largest element of 2D array
4. Write a program to find minimum element.
5. Write a program to print row sum.
6. Write a program to print sum of column.
7. Write a program to print diagonal elements
8. Write a program to Transpose of matrix.





Day- 10

Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR





ArrayList

Q. What is an ArrayList?

Answer: An ArrayList in Java is a resizable array that is part of the Java Collections Framework.

- A normal array has a fixed size.
- An ArrayList can grow or shrink dynamically according to the number of elements.
- It stores elements in an ordered manner and allows duplicate values.
- ArrayList is available in the java.util package.

Example:

```
ArrayList<String> list = new ArrayList<>();

list.add("Java");
list.add("Python");

System.out.println(list);
```



Array list

❖ **Important Points:**

- Dynamic size
- Easy insertion and deletion
- Maintains insertion order
- Allows duplicate elements

❖ **Import Statement:**

```
import java.util.ArrayList;
```

Example: Imagine a classroom attendance sheet.

- If you use an array of size 50, only 50 students can be stored.
- If new students join, the array size cannot increase easily.
- But an ArrayList can extend automatically when more students are added.

Note: An ArrayList is useful when the number of elements is not fixed because it grows and shrinks dynamically.



Array list

❖ Syntax of ArrayList in Java

1. For Homogeneous Data (Same Type of Data):

```
ArrayList<DataType> listName = new ArrayList<>();
```

Example:

```
ArrayList<Integer> numbers = new ArrayList<>();
```

- Used when all elements are of the same type.
- Provides type safety.

2. For Heterogeneous Data (Different Types of Data)

```
ArrayList list = new ArrayList();
```

Example:

```
ArrayList list = new ArrayList();
list.add(10);
list.add("Java");
list.add(true);
```

- Used when storing different types of data together.
- Type safety is not maintained.





Array list

Example:

```
ArrayList<Integer> numbers = new ArrayList<>();
```

Here:

ArrayList ---- Class name
Integer ---- Data type of elements
numbers ----- Variable name
new ArrayList<>() ----- Creates an ArrayList object





Array list

❖ **Common ArrayList Methods in Java**

1. add() → Adds an element
   list.add(10);
2. get() → Access an element
   list.get(0);
3. set() → Update an element
   list.set(0, 50);
4. remove() → Remove an element
   list.remove(0);
5. size() → Returns total number of elements
   list.size();
6. contains() → Checks element exists or not
   list.contains(10);
7. clear() → Removes all elements
   list.clear();
8. isEmpty() → Checks list is empty or not
   list.isEmpty();





Array list

❖ **Example:**

```
import java.util.ArrayList;
public class Demo {
        public static void main(String[] args) {
                ArrayList<Integer> list = new ArrayList<>();

                list.add(10);
                list.add(20);
                System.out.println(list);
                System.out.println(list.get(0));

                list.set(1, 50);

                System.out.println(list);
                list.remove(0);

                System.out.println(list);
        }
}
```





Day- 13

Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR





Today’s Agenda

- Sorting Fundamentals
- Linear Search Overview
- Binary Search Overview





Sorting Fundamentals

Q. What is Sorting?
Ans: Sorting means arranging data in a particular order.

- **Ascending order :-** Small to large

  Example: 1, 2, 3, 4, 5

- **Descending Order :-** Large to small .

  Example: 5, 4, 3, 2, 1

Q. Why do we need Sorting?
Ans: Without sorting data takes more time.

Example: Finding roll no. 2 in : 8, 1, 9, 2, 5

- You may check every element.





Sorting Fundamentals

- With sorting, in sorted data like: 1, 2, 5, 8, 9
- Searching becomes faster.

❖ **Advantage of Sorting**

- Faster Searching
- Easy data Management
- Better Data Analysis
- Helps in Ranking
- Used in Real Applications





Sorting Fundamentals

❖ **Most Common Searching Algorithms**

- Linear Search
- Binary Search
- Jump Search
- Interpolation Search
- Hashing



Linear Search

Q. What is Linear Search?
Ans: Linear search means checking every element one by one to find the target element.

Q. How Linear Search works?
Ans.

Step1: Start from first element

Step2: Compare with target element

Step3: If matched ----Found

Step4: Else move to next element

Step5: Repeat until end.



Linear Search

❖ **Visualization**

Array: {5, 8, 2, 9, 1}
Target : 9

Check 5------No
Check 8------No
Check 2------No
Check 9------Yes Found





Linear Search

❖ **Example:**

```
class LinearSearch {
        public static void main(String[] args) {

                int[] arr = {10, 20, 30, 40, 50};
                int target = 30;

                for(int i = 0; i < arr.length; i++) {
                        if(arr[i] == target) {
                        System.out.println("Element Found");
                        }
                }
        }
}
```





Binary Search

❖ **Definition**

Binary search finds an element by checking the middle element and reducing the search area into half repeatedly.

➢ Note: Binary search works only on sorted arrays.

Q. How Binary Search works?

Ans. Steps

1. Find middle element
2. Compare with target
3. if target == middle ---- Found
4. if target >middle ---- Search Right Half
5. if target <Middle ---- Search left half
6. Repeat until found.





Binary Search

- Visualization

Array : { 1, 3, 5, 7, 9, 11 }

target : 9

Step 1: Middle =5

9>5: Move Right

Step 2: Middle =9

Found





Binary Search

❖ **Advantages of Binary Search**

- Very Fast
- Efficient for large data
- Fewer comparisons

❖ **Disadvantages of Binary Search**

- Array must be sorted
- Slightly harder than linear search





Sorting Fundamentals

❖ **Common Sorting Algorithms**

1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Merge Sort
5. Quick Sort



Bubble Sort

❖ **Definition**

➢ Bubble Sort is a simple sorting algorithm that repeatedly compares adjacent elements and swaps them if they are in the wrong order.

➢ After every pass, the largest element moves to the end of the array , just like a bubble rising to the top of water.

❖ **Advantages**

- Easy to learn
- Simple implementation

❖ **Disadvantages**

- Slow for large data

Insertion Sort

❖ **Definition**

➢ Insertion Sort is a sorting algorithm that picks one element at a time and places it in its correct position in the sorted part of the array.

❖ **Advantages**

- Simple to implement

Efficient for small or nearly sorted arrays

❖ **Disadvantages**

- Slow for large datasets

Merge Sort

❖ **Definition**

➢ Merge Sort is a sorting algorithm that divides the array into smaller parts, sorts them separately, and then merges them together.

➢ It follows the **Divide and Conquer** approach.

❖ **Advantages**

- Very fast for large datasets
- Stable sorting algorithm

❖ **Disadvantages**

- Uses extra memory

Quick Sort

❖ **Definition**

➢ Quick Sort is a fast sorting algorithm that uses a **pivot element** to divide the array into two parts:

- Elements smaller than pivot
- Elements greater than pivot

➢ Then it sorts both parts recursively.

➢ It also follows **Divide and Conquer** approach.

❖ **Advantages**

- Very fast in practice
- Uses less memory than Merge Sort

❖ **Disadvantages**

- Worst case can be slow
- Not stable



Day- 15

Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR





Today's Agenda

- Strings
- Standard Operations



String in Java

Q. What is String?

Ans.

- A String in Java is a sequence of characters used to store text like names, words, or sentences.
- String is used to store words and sentences in Java.

Example:

Name ----> "Rahul"

City ----> “Vadodara"

Message -----> "Hello students“

String in Java

❖ **Key Features of String**

1. Immutable : Once a String is created, it **cannot be changed.**

Example:

```
String s = "Hello";
s.concat(" World");

System.out.println(s); // Hello
```

➢ Because original string does not change.

2. Stored in String Pool: Java saves String in a special memory area called **String Pool.**

Example:

```
String a = "Java";
String b = "Java";
```

➢ Both point to same memory → saves space



String in Java

❖ **How to create String in Java?**

Method 1 :

```
String s1 = "Hello";
```

Method 2 :

```
String s2 = new String("Hello");
```





String in Java

❖ **Common String Methods in Java**

1. **length():** Returns number of characters in a string

```
String s = "Java";
System.out.println(s.length()); // 4
```

2. **charAt(index):** Returns character at a specific position

```
String s = "Java";
System.out.println(s.charAt(1)); // a
```

3. **concat():** Joins two strings

```
String s1 = "Hello";
String s2 = "World";

System.out.println(s1.concat(s2)); // HelloWorld
```





String in Java

4. toUpperCase(): Converts string to capital letters

```
String s = "java";
System.out.println(s.toUpperCase()); // JAVA
```

5. toLowerCase(): Converts string to small letters

```
String s = "JAVA";
System.out.println(s.toLowerCase()); // java
```

6. substring(start, end): Extract part of string

```
String s = "Programming";
System.out.println(s.substring(0, 4)); // Prog
```

7. contains(): Checks if text is present or not

```
String s = "Java Programming";
System.out.println(s.contains("Java")); // true
```





String in Java

8. replace(): Replaces characters or words

```
String s = "banana";
System.out.println(s.replace('a', 'o')); // bonono
```

9. equals(): Compares two strings (case-sensitive)

```
String a = "Java";
String b = "Java";

System.out.println(a.equals(b)); // true
```

10. equalsIgnoreCase(): Compares without checking case

```
String a = "Java";
String b = "java";

System.out.println(a.equalsIgnoreCase(b)); // true
```





Day- 16

Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR



StringBuilder in Java

Q. What is StringBuilder?

Ans. StringBuilder is a class in Java used to create and modify strings efficiently.

- Normally, Strings in Java are **immutable** (cannot be changed).
- Whenever we modify a String, a new object is created.

- To avoid creating many objects, Java provides:

  StringBuilder -----> Faster, not thread-safe
  StringBuffer ------> Thread-safe, slower



StringBuilder in Java

Q. Why do we use StringBuilder?

Problem with String

```
String s = "Hello";
s = s + " World";
s = s + " Java";
```

- Every time + is used:
- New object is created
- Memory waste
- Slower performance

- **To avoid the problem of creating multiple unnecessary objects in String, StringBuilder was introduced in Java.**





StringBuilder in Java

Example:

```
StringBuilder sb = new StringBuilder("Hello");

sb.append(" World");
sb.append(" Java");

System.out.println(sb);
```

Output:

Hello World Java

- Here the same object is modified.





StringBuilder in Java

❖ **Internal Working**

➢ StringBuilder stores characters in a **mutable array**.

So operations like:

- append()
- insert()
- delete()
- reverse()

are faster compared to String.





StringBuilder in Java

❖ **Creating StringBuilder**

1. Empty StringBuilder

```
StringBuilder sb = new StringBuilder();
```

2. With Initial String

```
StringBuilder sb = new StringBuilder("Hello");
```

3. With Capacity

```
StringBuilder sb = new StringBuilder(50);
```

➢ **Capacity means initial storage size.**





StringBuilder in Java

❖ **Important Methods**

1. append(): Adds data at the end.

```
StringBuilder sb = new StringBuilder("Java");

sb.append(" Programming");
System.out.println(sb);
```

Output: Java Programming

2. insert(): Adds data at specific index.

```
StringBuilder sb = new StringBuilder("Jva");

sb.insert(1, "a");
System.out.println(sb);
```

Output: Java





StringBuilder in Java

3. delete(): Deletes characters.

```
StringBuilder sb = new StringBuilder("Hello");

sb.delete(1, 3);
System.out.println(sb);
```

Output: Hlo

4. reverse(): Reverses string.

```
StringBuilder sb = new StringBuilder("Java");

sb.reverse();
System.out.println(sb);
```

Output: avaJ





StringBuilder in Java

5. charAt(): Gets character at index.

```
StringBuilder sb = new StringBuilder("Java");

System.out.println(sb.charAt(2));
```

Output: v

6. setCharAt(): Updates character.

```
StringBuilder sb = new StringBuilder("Jovo");

sb.setCharAt(1, 'a’);
System.out.println(sb);
```

Output: Javo





StringBuilder in Java

7. length(): Returns size.

```
StringBuilder sb = new StringBuilder("Java");

System.out.println(sb.length());
```

Output: 4

8. capacity(): Returns total capacity.

```
StringBuilder sb = new StringBuilder();

System.out.println(sb.capacity());
```

Output: 16





Day- 17

Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR



Introduction to Oops

❖ **Problem before Oops**

- In Procedural programming language ,
    - ➢ Code was written in functions only.
    - ➢ Data was not secure (Any one would change)
    - ➢ Difficult to manage large projects.
    - ➢ Code reuse was limited.

✓ If you build a big system (bank app, website) everything become messy and hard to maintain.

❖ **Why Oops was introduced?**

- Oops was introduced to solve these problems and make programming…
    - ➢ More Organized
    - ➢ Real-word Representation
    - ➢ Data Security
    - ➢ Code Reusability
    - ➢ Flexibility
    - ➢ Hide Complexity



Introduction to Oops

- OOP was introduced to manage software complexity by modeling real-world entities, making programs more reusable, scalable, robust, and easier to maintain.

❖ **Class in Java :**

A class is a blueprint/ template to create objects . It defines:

- Variables (Data)
- Methods (Functions)

Example:

```
Class Students{
        String name;
        int marks;

        void display(){
                System.out.println(name+ " " +marks);
        }
}
```

❖ Class : design of a car (not a actual car).





Introduction to Oops

- **Objects in java:**

  An object is an instance of a class(real entity).

Example:

```
public class Main{
    public static void main(String[] args){
        Student s1= new Studnets();// object creation

        s1.name= “Aman”;
        s1.marks=90;
        s1.display();
}
```

- Object---> Actual car created from design.



Introduction to Oops

❖ **Constructor in java:**

➢ A constructor is a special method used to initialize objects.

- Same name as class
- No return type
- Automatically called when object is created.

❖ **Types of Constructor**

1. Default Constructor
2. Parametrized Constructor
3. Copy Constructor



Introduction to Oops

1. **Default Constructor:**

```
Class Student {
        Student (){
        System.out.println("Default Constructor Called");
        }
}
```

2. **Parameterized Constructor**

```
Class Student {
        String name;
        Student(String n){
                name= n;
        }
}
```





Introduction to Oops

3. Copy Constructor:

```
Class Student{
        string name;
        Student(String n){
                name=n;
        }

        Student(Students){
                this.name = s.name;
        }
}
```





Day- 19

Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR





Encapsulation

Q. What is Encapsulation?

Ans. Encapsulation means wrapping data and methods together into a single unit (class) and restricting direct access to the data. It helps in data hiding and provides controlled access through methods.

Example: Think of an ATM machine.

- You cannot directly access your bank balance. Instead, you interact with it using methods like:
  - withdraw()
  - deposit()
  - checkBalance()
- The actual data (balance) is hidden, and you interact with it through controlled methods only.





Encapsulation

Q. How is Encapsulation achieved in Java?
Ans. There are 2 main steps:

1. **Make variables private**

```
private int balance;
```

2. Provide public methods (getter/setter)

```
public int getBalance() {
        return balance;
}

public void setBalance(int balance) {
        this.balance = balance;
}
```





Encapsulation

Example:

```
Class BankAccount(){
        private int balance; // hidden data

        //setter method
        public void setBalance(int balance){
                if(balance>=0){
                        this.balance=balance;
                }else{
                        System.out.println("Invalid Amount");
                }
        }

        //getter Method
        public int getBalance(){
                return balance;
        }
}
```





Encapsulation

```
public class Main(){
        public static void main(String[] args){

                BankAccount obj= new BankAccount();
                obj.setBalance(5000);
                System.out.println(obj.getBalance());
        }
}
```

Q. Why Encapsulation is Important?

Ans.

1. **Data Security:** No direct access to variable.
2. **Control:** You can add validation(like a negative balance).
3. **Flexibility:** Cleaner and Structured code.
4. **Code Maintainability:-** Cleaner and Structured code.



Inheritance

❖ **Definition:**

➢ One class acquires the properties and behaviours of another class. It provides code reuse.

➢ Inheritance is a mechanism where once class(Child) inherits the properties and methods of another class(parents).

❖ **Example:** Think of a parent and child.

- Child inherits
- Surname
- Habit
- Properties

➢ Child class gets variables and methods from parents class.



Inheritance

❖ **Syntax:**

```
Class Parents{

        //properties and method
}

class child extends Parents{

        //inherits everything from parents.
}
```





Inheritance

❖ **Example:**

```
Class Animal{
        void eat(){
            System.out.println("Animal is eating ");
        }
}
Class dog extends Aniaml{
        void bark(){
            System.out.println("Dog is barking ");
        }
}
public class Main{
        public static void main(String[] args){
                Dog d= new Dog();
                d.eat();// inherited from Animal
                d.bark(); // own method
        }
}
```



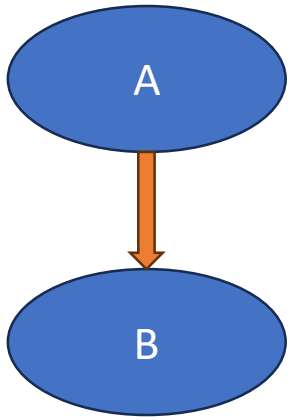
Inheritance

❖ **Types of Inheritance in Java**

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. multiple Inheritance

1. Single Inheritance:

Single inheritance in java means one child class inherits from one parents class.





Inheritance

❖ **Example:**

```
Class Animal{
        void eat(){
            System.out.println("Eating…..");
        }
}

Class Dog extends Animal{
        void bark(){
            System.out.println("Barking…..");
        }
}
```

➢ Dog inherited from Animal .

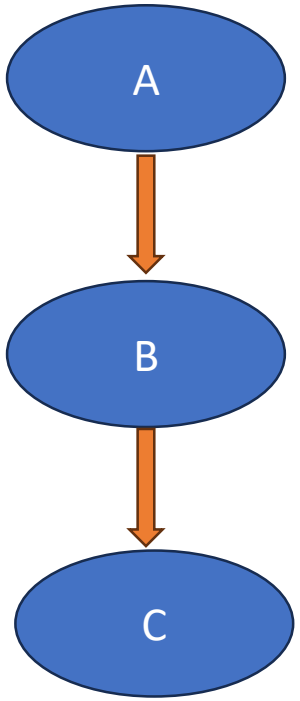


Inheritance

2. Multilevel Inheritance:

Multilevel inheritance in java means a class inherits from a class and that class also inherits from another class.

Grandparent -----> Parent-----> Child





Inheritance

❖ **Example:**

```
Class Animal{
        void eat(){
            System.out.println("Eating…..");
        }
}

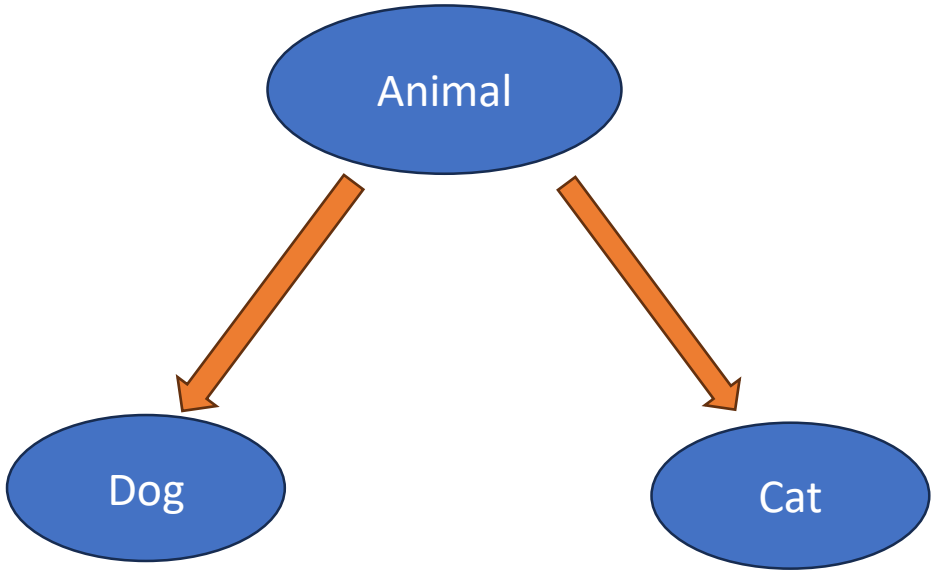
Class Dog extends Animal{
        void bark(){
            System.out.println("Barking…..");
        }
}
Class puppy extends Dog{
        void weep(){
            System.out.println("Weeping….");
        }
}
```



Inheritance

2. Hierarchical Inheritance:

Hierarchical inheritance in java means multiple child classes inherit from one parent class.



- Both inherit common properties from Animal.



Inheritance

❖ **Example:**

```
Class Animal{
        void eat(){
            System.out.println("Eating…..");
        }
}

Class Dog extends Animal{
        void bark(){
            System.out.println("Barking…..");
        }
}
Class Cat extends Animal{
        void meow(){
            System.out.println("Cat is meowing….");
        }
}
```





Inheritance

```
Public class Main{
        public static void main(String[] args){
                Dog d= new Dog();
                Cat c= new Cat();

                d.eat();// inheritaed
                d. bark();//

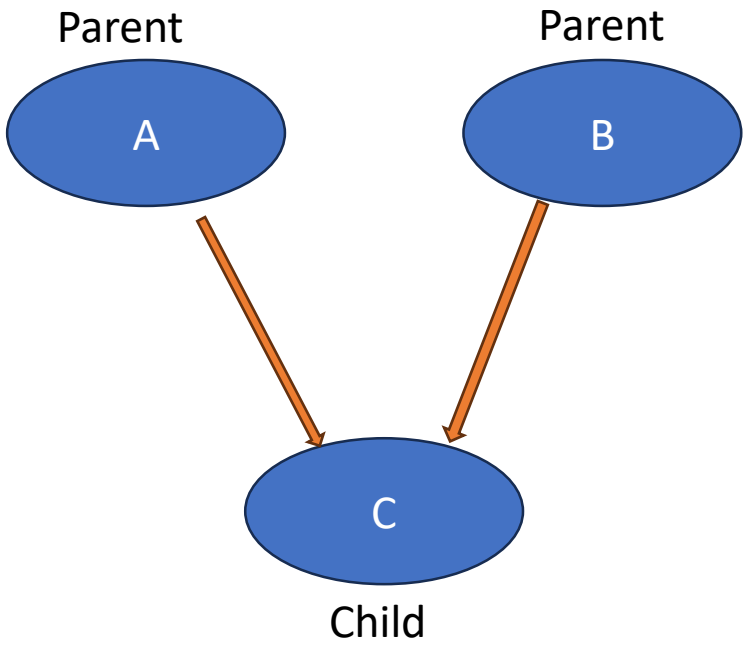
                c.eat();// inheriated
                c.meow();//
        }
}
```



Inheritance

4. Multiple Inheritance:

Multiple inheritance means one child class inherits from more than one parents class.



- It means class C wants properties from both A and B.



Inheritance

- **Note:** Java does not support multiple inheritance with classes because it can create ambiguity(Called the Diamond problem).

Example:(Not Allowed)

```
Class A{
        void show(){
                System.out.println("A");
        }
}
Class B{
        void show(){
                System.out.println("B");
        }
}
Class C extends A, B{ //Not allowed in java.

}
```

- If both A and B have the same method(Show()), java gets confused about which one C should use.





Inheritance

❖ **We can achieve multiple inheritance by using interface.**

Example:

```
Interface  A{
            void show();
}
Interface  B{
            void  display();
}
Class C implements A, B{
            public void show(){
                        System.out.println(“Show Method”);
            }

            public void display(){
                        System.out.println(“Display Method”);
            }
}
```





Inheritance

```
Public class Main{
        public static void main(String[] args){
                C obj= new C();

                obj.show();
                obj.display();
        }
}
```





Polymorphism

❖ **Definition:**

➢ Polymorphism is a core concept of object oriented programming where a single entity(method, object or interface) can take multiple forms.

➢ Same method name but different behaviour.

❖ **Why Polymorphism:**

➢ Without polymorphism you would need different method names for every behaviour.

❖ **Without Polymorphism :**

```
dogSound();
catSound();
cowSound();
```

❖ **With Polymorphism:**

```
animal.sound();
```

✓ Leaner, scalable and easier to maintain.





Polymorphism

❖ **Types of Polymorphism.**

1. Compile Time Polymorphism

- Achieved using method overloading.
- Decision is made at compile time.

2. Rules of method overloading.

- Same method name
- Different parameters(type , number, order)
- Return type alone is not enough





Polymorphism

❖ **Example**

```
Class Calculator{
        int add(int a, int b){
                return a+b;
        }

        int add(int a, int b, int c){
                return a+b+c;
        }
        double add(double a, double b){
                return a+b;
        }
}
```

➢ Here compiler decides which method to call.





Polymorphism

2. Runtime Polymorphism

- Achieved using method overriding.
- Decision is made at runtime.(JVM Decides)

❖ Rules of method overriding.

- Same method name
- Same parameters
- Must have inheritance
- Method is child class overrides parents class.





Polymorphism

❖ **Example**

```
Class Animal{
        void Sound(){
                System.out.println("Animal makes sound");
        }
}

Class Dog extends Animal{
        void Sound(){
                System.out.println("Dog Bark");
        }
}

Class Cat extends  Animal{
        void Sound(){
                System.out.println("Cat Mews");
        }
}
```



Polymorphism

```
Animal a;
A=new Dog();

a.Sound(); // Dog barks

A=new Cat();
a.Sound();//  cat meows.
```

❖ **How Polymorphism works?**

- Reference variable is created(Animal)
- Object is assigned (Dog)
- Method call is made ---- a.sound()
- JVM checks actual objects---- Dog
- Calls Dog's method.



Polymorphism

Example:

```
Class Payment{
        void pay(){
            System.out.println("Generic Payment");
        }
}
Class upi extends Payment{
        void pay(){
            System.out.println("Pay using Upi");
        }
}

Class card extends Payment{
        void pay(){
            System.out.println("Pay using Card");
        }
}
```

- **Same method pay()--- different behaviour .**



Abstraction

❖ **Definition**

- Abstraction in java means hiding implementation details and showing only essential features.
- User only knows what to do, not how it works.

Example: Think of ATM

You only know….

- Insert Card
- Enter PIN
- Withdraw money

- But you don’t know the internal banking process.
- That hidden internal process is abstraction.

In Java, abstraction is achieved by:

- Abstract class
- interface



Abstraction

❖ Abstract class: An abstract class is a class that can't be instantiated(cannot create object directly).

It can have:

- Abstract method(Without body)
- Normal methods(With Body)

Example:

```
abstract class Animal{
        abstract void sound(){
                System.out.println("Animal eats");
        }
}
Class Dog extends Animal{
        void  sound(){
                System.out.println("Dog barks");
        }
}
```





Abstraction

```
public class Main{
        public static void main(String[] args){
                Dog d= new Dog();
                d.sound();
                d.eat();
        }
}
```

Output:

```
        Dog Barks
        Animal eats
```





Abstraction

2. Interface:

Interface gives 100% abstraction(Conceptually)

Example:

```
interface Vehicle{
        void start();
}
Class car implemnets Vehicle{
        public void start(){
                System.out.println(“Car Start”);
        }
}
Public class Main{
        public static void main(String[] args){
                Car c= new Car();
                c.start();
        }
}
```





Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR





Recursion in Java

Q. What is Recursion?

Ans. Recursion is a technique where a method calls itself again and again to solve a problem. It is mainly used when a problem can be divided into smaller similar problems.

❖ **Basic Structure of Recursion**

```
returnType methodName() {

        // Base Condition
        if(condition) {
        return value;
 }

        // Recursive Call
        return methodName();
}
```





Recursion in Java

❖ **Important Terms**

1. Base Case

- The condition where recursion stops.
- Without a base case, recursion will run forever and cause:

StackOverflowError

2. Recursive Call

- The method calling itself.

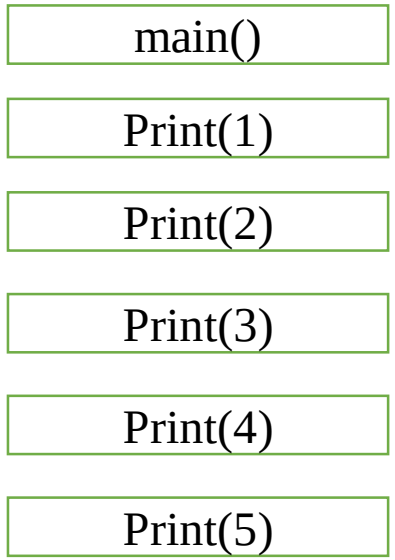


Recursion in Java

Q. Why Recursion?
Ans.

- It helps us in solving bigger/complex problems in a simple way.
- You can convert recursive solution into iteration and vice versa.
- Space complexity is not constant because of recursive call.
- It helps us in breaking down bigger problem into smaller problems.

❖ **Visualization of Recursion**





Recursion in Java

Example:

```
class RecursionDemo {

    static void printNum(int n) {
        // Base case
        if(n > 5) {
        return;
        }
        System.out.println(n);
        // Recursive call
        printNum(n + 1);
        }
        public static void main(String[] args) {
        printNum(1);
    }
}
```





Impact Training Program

PREPARED BY: ASHWANI MISHRA | ASSISTANT PROFESSOR





Linked List in Java



Linked List in Java

Q. What is a Linked List?

Ans. A Linked List is a linear data structure where elements are stored in separate nodes. Each node contains:

- Data (the value)
- Address/Reference of the next node

- First node = Head
- Last node points to NULL

Example:

Imagine a train:

- Each coach is connected to the next coach.
- If a new coach is added, it can be attached without moving other coaches.



Recursion in Java

Q. Why Do We Need Linked Lists?

Problem with Arrays:

- Fixed size
- Insertion and deletion are costly because elements need to be shifted

Solution: Linked List

- Dynamic size
- Easy insertion and deletion

❖ **Types of Linked Lists**

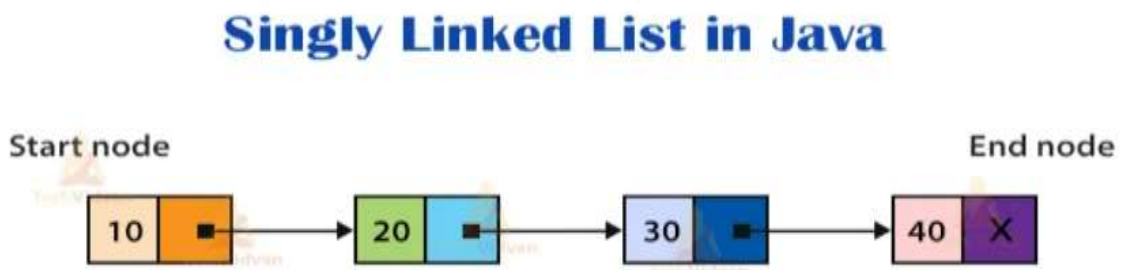
1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List



Singly Linked List

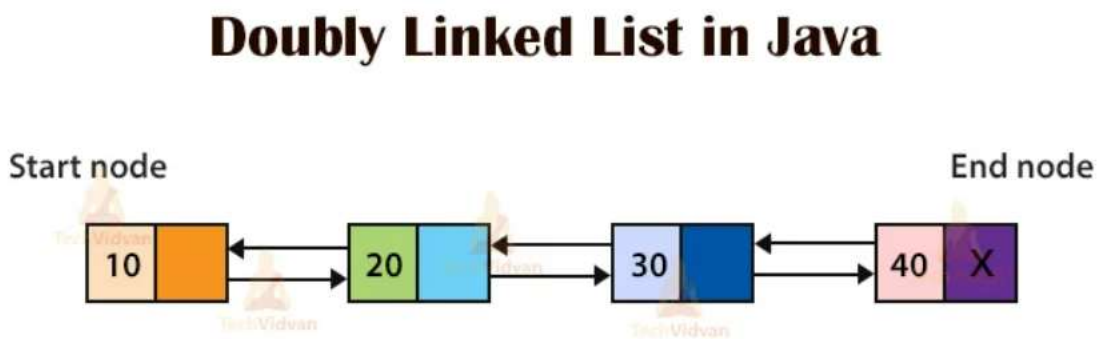
Q. What is Singly Linked List?

Ans. Each node points to the next node.



Doubly Linked List

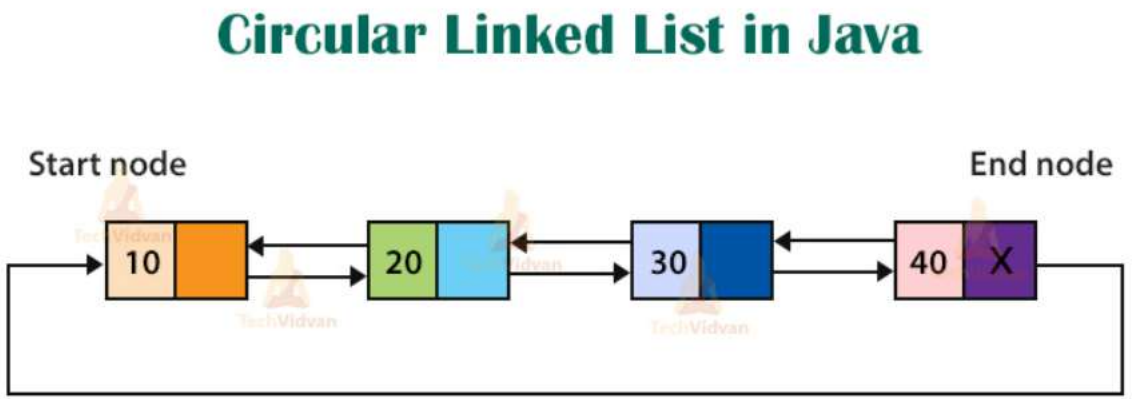
Q. What is Doubly Linked List?
Ans. Each node has pointers to both previous and next nodes.



Circular Linked List

Q. What is Circular Linked List?

Ans. The last node points back to the first node.





Basic Operations of Linked List

❖ **Basic Operations of LinkedList**

1. **Insertion :** Add a new node to the list.
2. **Deletion :** Remove a node from the list.
3. **Traversal :** Visit each node one by one.
4. **Searching :** Find a particular element in the list.





Java LinkedList Class

Example:

```
import java.util.LinkedList;

public class Demo {
        public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();

        list.add("Java");
        list.add("Python");
        list.add("C++");

        System.out.println(list);
        }
}
```

Output: [Java, Python, C++]





Java LinkedList Class

❖ **Common Methods**

Method	Description
add()	Add element
addFirst()	Add at beginning
addLast()	Add at end
remove()	Remove element
removeFirst()	Remove first element
removeLast()	Remove last element
get()	Get element by index
size()	Number of elements




Thank You!